Table of Contents

MAE 6760 Model Based Estimation 1
User inputs 1
--------- start-up items 7

MAE 6760 Model Based Estimation

Cornell University M Campbell

Homework #4 Problem #2: Sigma Point Filter (SPF) two state Van der Pol oscillator nonlinear system Plotting uses plot_openloop.m and plot_estimator.m

```
close all;
clear all;
MAE6760startup; %can adjust font size, figure size at the bottom of this
script
global MCcolors; %define colors as global with access
rng(10);
%
```

User inputs

```
%van der pol constant
mu=[0.1,5];
%mu = 0.1;

int_method = "euler"; %alternate = ode45
%int_method = "ode45";

%function files;
if int_method == "ode45"
    ffun='predict_state_vdp'; %m-file for Xk+1=f(Xk,Uk);
else
    ffun='predict_state_vdp_euler'; %m-file for Xk+1=f(Xk,Uk);
end

hfun='predict_msmt_vdp'; %m-file for Zk+1=h(Xk+1);

nsig=[0.1, 0.5, 1];
nsig = 0.1;

for i = 1:length(mu)
    for j = 1:length(nsig)


        curr_mu = mu(i);
        curr_nsig = nsig(j);
        %initialization
        P0_vals = [0.5^2, 0.5^2];
```

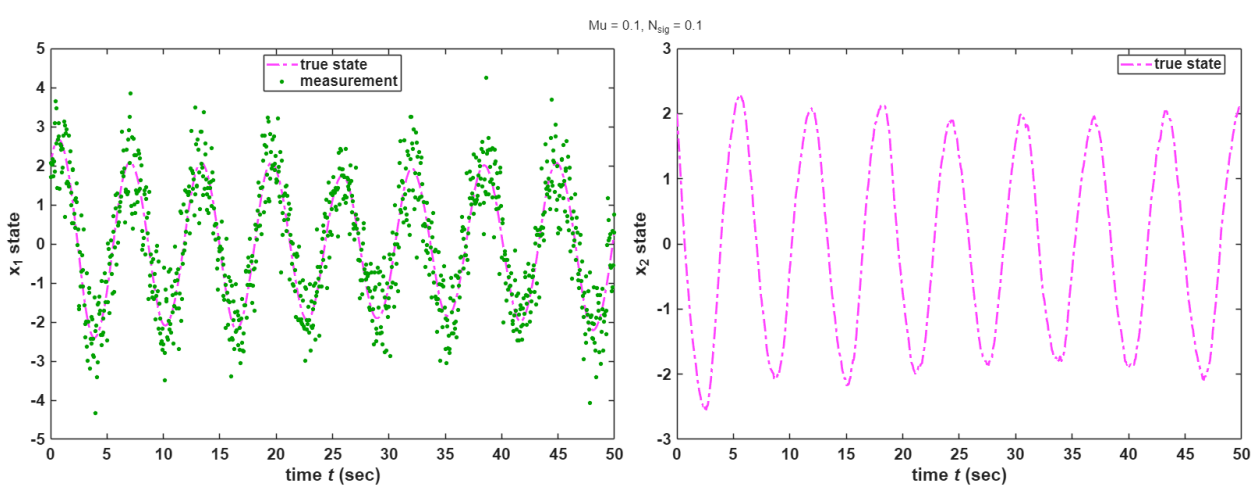
```
        P0=diag(P0_vals);
        %
        %
        %
        %simulate input process noise
        dt=0.05;tf=50;
        t=[0:dt:tf];
        nt=length(t);
        %
        Qc=0.01;
        w=randn(nt,1)*[sqrt(Qc)/sqrt(dt)]; %discrete approximation to CT
white noise
        Qd=Qc/dt; %discrete time white noise covariance for the SPF
        G=[0;dt]; %mapping of disturbance B to state [x1;x2]

        %simulate the van der pol oscillator via ODE 45
        x0=[2;2];
        nx=2; %number of states
        x_true=x0;
        for k=1:(nt-1),
            x_init=x_true(:,k);
            wk=w(k);
            tspan=[t(k) t(k+1)];
            [ttmp,x_ode45]=ode45(@(t,x)
vanderpol(tspan,x,curr_mu,wk),tspan,x_init);
            x_true(:,k+1)=x_ode45(end,:)';
        end

        %create noisy measurement
        R=0.5;
        v=sqrt(R)*randn(1,nt);
        z=x_true(1,:)+v;

        %open loop plots
        figure('Position',[100 100 1600 600]);
        title("test")
        tcl = tiledlayout(1,2,'TileSpacing','compact','Padding','tight');
        nexttile;
        iix=1;
        plot_openloop(t,x_true(iix,:),z)
        grid
        ylabel('x_1 state');
        nexttile;
        iix=2;
        plot_openloop(t,x_true(iix,:))
        grid
        ylabel('x_2 state');
        title(tcl,"Mu = " + string(curr_mu) + ", N_s_i_g = " +
string(curr_nsig));


        %Sigma Point Filter (SPF)

        %initialization
```

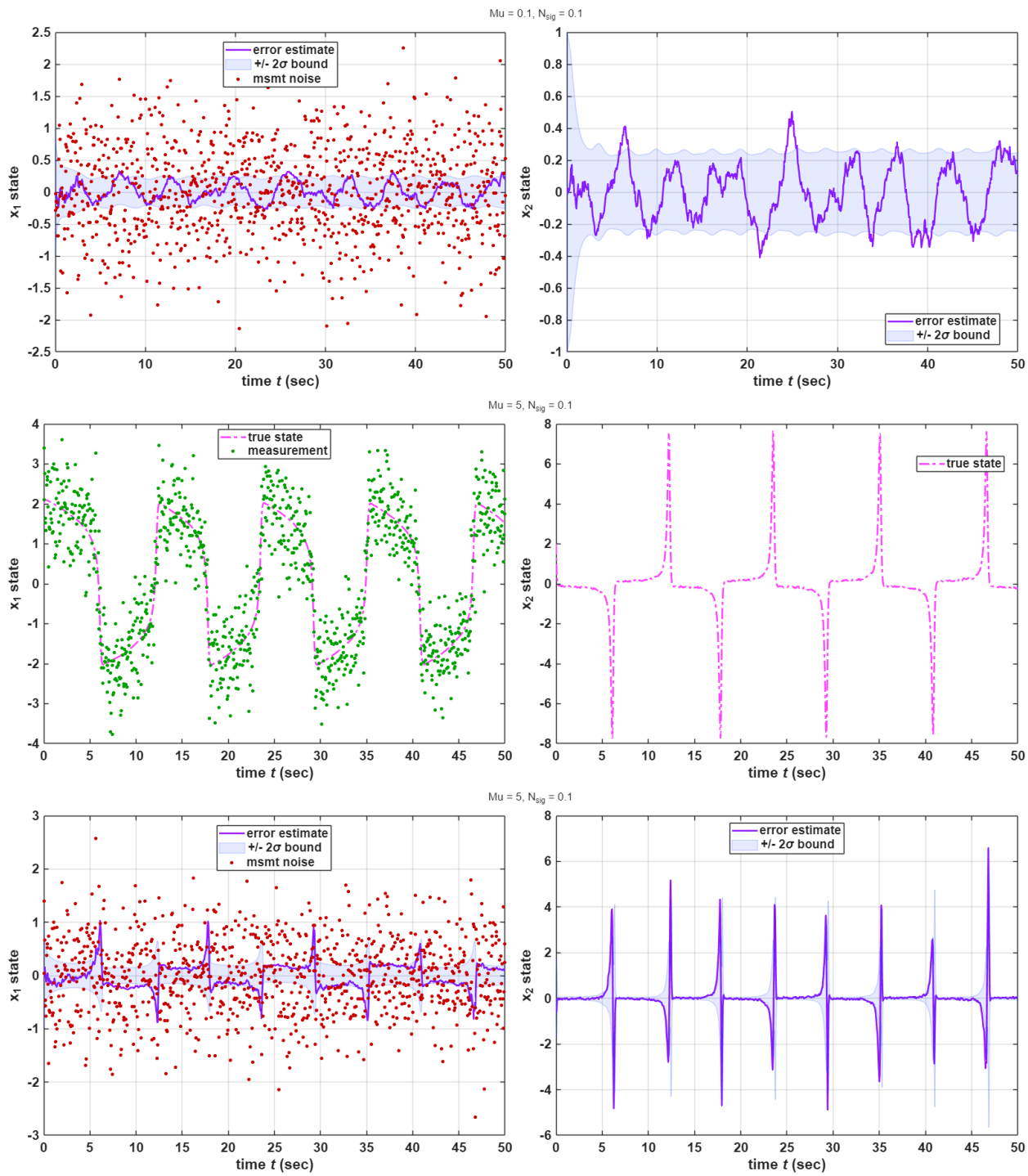
```
        xhatp=zeros(nx,nt);xhatp(:,1)=x0;
        Pp=zeros(nx,nx,nt);Pp(:,:,1)=P0;
        xhatu=xhatp;
        Pu=Pp;
        for k=1:(nt-1)
            zkp1=z(k+1);
            Qx=G*Qd*G';
            [xhatu(:,k+1),Pu(:,:,k+1),xPred,zPred,innovation]=...
                spf_vdp(xhatu(:,k),Pu(:,:,k),
[],Qx,ffun,zkp1,R,hfun,t(k),t(k+1),curr_mu,curr_nsig);
        end

        figure('Position',[100 100 1600 600]);
        tcl = tiledlayout(1,2,'TileSpacing','compact','Padding','tight');
        nexttile;
        iix=1;

plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error',z(1,:))
        ylabel('x_1 state');
        iix=2;
        nexttile;
        plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error')
        ylabel('x_2 state');
        title(tcl,"Mu = " + string(curr_mu) + ", N_s_i_g = " +
string(curr_nsig));
    end
end

function xdot=vanderpol(t,x,mu,w);
%
%   nonlinear differential equations for the van der pol oscillator
%
xdot1 = x(2);
xdot2 = -x(1) + mu*(1-x(1)^2)*x(2) + w;
xdot=[xdot1;xdot2];
%
end

function Xkp1=predict_state_vdp(Xk,mu,tk,tkp1);
%
%   For the van der pol oscillator
%   discrete prediction of state at k+1
%   via ode 45
%
%   mu is the constant from the van der pol oscillator
%   tk is the current time
%   tkp1 is tk+dt
%
%   Xk is an (nx x nsigpoints) matrix (need to be careful with this!)
%
[nx,nsp]=size(Xk);
%
for i=1:nsp,
    xinit=Xk(:,i);
```

```
    wk=0;
    tspan=[tk tkp1];
    [~,x]=ode45(@(t,x) vanderpol(tspan,x,mu,wk),tspan,xinit);
    Xkp1(:,i)=x(end,:)';
end
%
end


function Xkp1=predict_state_vdp_euler(Xk,mu,tk,tkp1);
%
%   For the van der pol oscillator
%   discrete prediction of state at k+1
%   via Euler integration
%
%   mu is the constant from the van der pol oscillator
%   tk is the current time
%   tkp1 is tk+dt
%
%   Xk is an (nx x nsigpoints) matrix (need to be careful with this!)

x1 = Xk(1,:);
x2 = Xk(2,:);
dt = tkp1-tk;
[nx,nsp]=size(Xk);

Xkp1 = Xk +...
    dt*[x2;
    -x1 + mu.*(1-(x1.^2)).*x2];

end


function Zkp=predict_msmt_vdp(Xkp1,dt);
%
%   For the van der pol oscillator
%   discrete prediction of measurement zhat at k+1
%
%   Xkp1 is an (nx x nsigpoints) matrix (need to be careful with this!)
%
H=[1 0];
Zkp = H*Xkp1;
end


function
[xEst,PxEst,xPred,zPred,innovation]=spf_vdp(xEst,PxEst,U,Q,ffun,z,R,hfun,tk,t
kp1,mu,nsig);
%
%   SPF with additive noise: passes mu for VDP oscillator
%
% SIGMA POINT FILTER
% One iteration of SPF, including prediction and correction.
% Assumes additive process and sensor noise
```

```
%
%
[xEst,PxEst,xPred,zPred,innovation]=spf_vdp(xEst,PxEst,U,Q,ffun,z,R,hfun,dt,n
sig);
%
% INPUTS   :  - xEst             : state mean estimate at time k
%             - PxEst            : state covariance at time k
%             - U                : vector of control inputs
%             - Q                : process noise covariance at time k
%             - ffun             : process model function (.m file)
%             - z                : observation at k+1
%             - R                : measurement noise covariance at k+1
%             - hfun             : observation model function (.m file)
%             - dt               : time step (passed to ffun/hfun)
%                - nsig             : sigma point scaling factor. Defaults
to 0.5.
%
% OUTPUTS  :  - xEst             : updated estimate of state mean at time k+1
%              - PxEst            : updated state covariance at time k+1
%             - xPred            : prediction of state mean at time k+1
%             - PPred            : prediction of state covariance at time k+1
%              - innovation       : innovation vector
%
% AUTHORS  :  Mark Campbell         (mc288@cornell.edu) 2003
%
% DATE     :  15 Oct 2003
%
% NOTES    :  This code was written to be readable. There is significant
%             scope for optimization even in Matlab.
%

%-----INITIAL CONSTANTS-----------------------------------------------------
nx = length(xEst);  %number of states
nsp=2*nx+1;        %number of sigma points
ensp=ones(1,nsp); %vector of all ones
%
%-----GENERATE WEIGHTING MATRICES-----------------------------------------
Wi=0.5/nsig^2;
W0M=(nsig^2-nx)/nsig^2;
W0C=(nsig^2-nx)/nsig^2+3-nsig^2/nx;
%vector form
WM=[W0M;ones(2*nx,1)*Wi];
WC=[W0C;ones(2*nx,1)*Wi];
%-------------------------------------------------------------------------

%%-----INITIALIZE-----------------------------------------
Psqrtm = nsig*chol(PxEst)';
xSigmaPts=[zeros(nx,1) -Psqrtm Psqrtm];
xSigmaPts = xSigmaPts + xEst*ensp;
%-------------------------------------------------------------------------

%%-----PREDICTION OF SIGMAPOINTS-------------------------------------------
%
%%%%%% Propagate state sigma points through dynamics
```

```
xPredSigmaPts = feval(ffun,xSigmaPts,mu,tk,tkp1);

%%%%%% Calculate Mean (a priori)
xPred = xPredSigmaPts*WM;

%%%%% Calculate Covariance (a priori)
exSigmaPts = xPredSigmaPts - xPred*ensp;
PxxPred = exSigmaPts*[diag(WC)]*exSigmaPts' + Q;

%%-----MEASUREMENT UPDATE OF SIGMAPOINTS-----------------------------------
%
%%%% INITIALIZE SIGMA POINTS AGAIN FOR ADDED Q CASE
Psqrtm = nsig*chol(PxxPred)';
exSigmaPts=[zeros(nx,1) -Psqrtm Psqrtm];
xPredSigmaPts = exSigmaPts + xPred*ensp;

%%%% Evaluate predicted state sigma points through output
zPredSigmaPts = feval(hfun,xPredSigmaPts);

%%%%%% Calculate Measurement Mean
zPred = zPredSigmaPts*WM;

%%%%%% Calculate Kalman Gain
ezSigmaPts = zPredSigmaPts - zPred*ensp;
PxzPred = exSigmaPts*[diag(WC)]*ezSigmaPts';
Pyy = ezSigmaPts*[diag(WC)]*ezSigmaPts' + R;
K = (PxzPred)*inv(Pyy);

%%%%% Update Covariance
PxEst = [PxxPred - K*PxzPred'];

%%%%% Calculate Innovation
innovation = z - zPred;
%%%%%% Update mean
xEst = xPred + K*innovation;
end
```





--------- start-up items

```
function MAE6760startup(font_size);
%
% define colors for plotting
global MCcolors; %define colors as global with access
MCcolors.red=[200,0,0]/255;
```

```
MCcolors.blue=[4,51,255]/255;
MCcolors.purple=[147,23,255]/255;
MCcolors.green=[0,160,0]/255;
MCcolors.orange=[253,128,8]/255;
MCcolors.mag=[255,64,255]/255;
MCcolors.cyan=[0,230,255]/255;
%
% define standard figure positioning and size
set(groot,'DefaultFigureUnits','pixels');
set(groot,'DefaultFigurePosition',[100 100 800 600]);
set(groot,'DefaultFigureWindowStyle','normal');  % Important
set(groot,'DefaultAxesFontSize',16);
set(groot,'DefaultAxesFontWeight','bold');
set(groot,'DefaultLineLineWidth',2);
%
end
```

Published with MATLAB® R2025b